

**HƯỚNG DẪN GIẢI MỘT SỐ CÂU
TRONG ĐỀ THI CHUNG KẾT ICPC PTIT NĂM 2024**

BÀI A. BIỂU THỨC TOÁN HỌC

- Ta cần đặt $N - 1$ phép toán vào giữa N số
- $N = 10 \rightarrow 9$ vị trí. Mỗi vị trí có 3 cách đặt toán tử, do đó, số trường hợp cần xét là $3^9 = 19683 \sim 2e4$. Giới hạn bé như vậy thì đây là một bài sinh kế tiếp/quay lui cơ bản. (Thực chất là sinh chỉnh hợp lặp chập 3 của $(N - 1)$ phần tử)
- Tuy nhiên: $a[i]$ lên đến 10^9 , chỉ cần nhân liên tục 3 số đã lên đến 10^{27} , vượt quá giới hạn kiểu dữ liệu long long trong C++. Nên tính đến đâu phải mod đến đấy. Chú ý rằng phép cộng và phép nhân cũng có tính phân phối trong modulo.
- Sau khi ghép xong dấu vào $N - 1$ vị trí rồi, ta sẽ được một biểu thức trung tố như dạng hay viết hàng ngày. Nhưng máy tính không thể hiểu được biểu thức trung tố, bắt buộc ta phải chuyển về biểu thức tiền tố hoặc hậu tố rồi dùng stack để giải
- Về nguyên tắc chung là như vậy, nhưng rất may đây là biểu thức không có dấu ngoặc, nên ta chỉ cần nhân trước, cộng trừ sau. Khi tính tay, ta sẽ thực hiện hết mọi phép nhân, được bao nhiêu cộng dồn lại với nhau. Dựa trên việc này, không cần dùng stack xử lý, ta sẽ làm như sau:
 - Biến res lưu kết quả cuối cùng
 - Biến tmp: Lưu kết quả của phép nhân gần nhất
- Cách thực hiện: Ta sẽ duyệt qua từng phép toán trong dãy phép toán $+$, $-$, $*$ mà ta đã sinh/quay lui (Tương đương với việc duyệt các phần tử $a[2] \dots a[N]$ cùng với dấu đứng trước nó).
 - Nếu trước $a[i]$ là phép nhân thì nhân tiếp $a[i]$ vào tmp
 - Nếu trước $a[i]$ là phép toán $+$ hoặc $-$: Lúc này, dãy nhân đã bị gãy, nên ta cộng dồn kết quả phép nhân gần nhất vào res đồng thời reset lại phép nhân thành $+a[i]$, nếu trước $a[i]$ là dấu $+$, hoặc $-a[i]$ nếu trước $a[i]$ là dấu $-$
- Thuật toán chuyển từ trung tố về hậu tố hoặc tiền tố khá dài, nhờ việc xử lý khéo léo ta đã tránh được việc này.

BÀI B. SỐ MAY MẮN ĐẶC BIỆT

- Khi so sánh 2 số dưới dạng xâu (Trường hợp không có số 0 thừa ở đầu):
 - Số nào ít chữ số hơn thì số đó là số nhỏ hơn
 - 2 số cùng số chữ số, số nào có thứ tự từ điển đứng trước thì nhỏ hơn
- Ý tưởng: Ta sẽ duyệt theo độ dài, cố gắng độ dài ngắn nhất có thể. Trong mỗi độ dài ta xét, cố gắng số chữ số 6 nhiều nhất có thể
- Chuẩn bị trước 2 mảng: $\text{pow}[i] = 10^i \% m$, $\text{one}[i] = (11\dots1 \rightarrow i \text{ số } 1) \% m$
- Giả sử số tìm được gồm a số 8, b số 6. Khi đó:
 - Giá trị của a số 8 là $A = 8 * \text{one}[a]$, giá trị của b số 6 là $B = 6 * \text{one}[b]$
 - Khi ghép lại thành số hoàn chỉnh, a số 8 phải thêm b số 0 ở cuối để nhường chỗ cho b số 6. Mà thêm b số 0 ở cuối chính là nhân với 10^b .
 - Như vậy, số hoàn chỉnh khi lấy dư cho m sẽ là:
$$R = A * 10^b + B = 8 * \text{one}[a] * \text{pow}[b] + 6 * \text{one}[b]$$
 - Nếu $R = 0$, đây là số cần tìm
- Triển khai thuật toán:
 - Tiên xử lý mảng one và pow như trên; Duyệt độ dài từ 1 đến 200
 - Với mỗi độ dài, duyệt số chữ số 8 = a từ 0 đến 200 (Ưu tiên a ít để b = len - a nhiều để số có thứ tự từ điển nhỏ)
 - Tính A, B, R như trên. Nếu thỏa mãn thì in a số 8 và b số 6 liền nhau.
 - Tổng số trường hợp tối đa cần xét với toàn bài:
 $100 \text{ (Số Test)} * 200 \text{ (Số cách chọn độ dài)} * 200 \text{ (Số cách chọn số chữ số 8)} = 4.10^6$, đủ nhanh trong 1s

BÀI C. TÍCH CỦA DÃY SỐ

Yêu cầu: Tìm số lượng dãy số có N phần tử mà tích cả dãy đúng bằng M

- $M = p_1^{x1} * p_2^{x2} * \dots * p_t^{xt}$
- Ta cũng phân tích từng số trong dãy ra thừa số nguyên tố, từ đó biểu diễn theo các thừa số của M như sau:
 - $a[1] = p_1^{a11} * p_2^{a12} * \dots * p_t^{a1t}$
 - $a[2] = p_1^{a21} * p_2^{a22} * \dots * p_t^{a2t}$
 - ...
 - $a[N] = p_1^{aN1} * p_2^{aN2} * \dots * p_t^{aNt}$
- Như vậy, tích các phần tử trong dãy là:
$$S = a[1] * a[2] * \dots * a[N]$$
$$= (p_1^{a11} * p_2^{a12} * \dots * p_t^{a1t}) * (p_1^{a21} * p_2^{a22} * \dots * p_t^{a2t}) * \dots * (p_1^{aN1} * p_2^{aN2} * \dots * p_t^{aNt})$$
- Rút gọn tích các lũy thừa cùng cơ số với nhau, ta có:
$$S = p_1^{a11 + a21 + \dots + aN1} * p_2^{a12 + a22 + \dots + aN2} * \dots * p_t^{a1t + a2t + \dots + aNt}$$
- Ta cần $S = M$, đồng nhất số mũ ta có:
 - $a11 + a21 + \dots + aN1 = x1 \text{ (1)}$
 - $a12 + a22 + \dots + aN2 = x2 \text{ (2)}$
 - ...
 - $a1t + a2t + \dots + aNt = xt \text{ (t)}$

Nghĩa là ta cần đếm số nghiệm nguyên không âm của t phương trình này

• **Xét một số bài toán cơ bản:**

Bài toán 1: Phương trình $x_1 + x_2 + \dots + x_k = N$ có bao nhiêu nghiệm nguyên dương

Giải: Ta viết số N thành N số 1 trên cùng 1 hàng: 1 1 ... 1 1 1

Cần chia N số 1 này thành k phần, mỗi phần có ít nhất 1 số 1

Giữa N số 1 có $N - 1$ khe trống, ta cần chọn k khe trống để đặt thanh chắn, nhằm chia thành k phần $\rightarrow C_{N-1}^{k-1}$

Bài toán 2: Phương trình $x_1 + x_2 + \dots + x_k = N$ có bao nhiêu nghiệm nguyên **không âm**?

Giải:

+) Ta sẽ biến đổi các số hạng vế trái thành số dương để quy về bài toán 1:

$$x_1 \geq 0 \rightarrow x_1 + 1 \geq 1 > 0.$$

+) Ta cộng vào mỗi số hạng vế trái thêm 1, phương trình trở thành:

$$(x_1 + 1) + (x_2 + 1) + \dots + (x_k + 1) = N + k$$

+) Đặt $x_1 + 1 = y_1, x_2 + 1 = y_2, \dots, x_k + 1 = y_k \rightarrow y_1 + y_2 + \dots + y_k = N + k$ (*)

+) Bài toán trở thành đếm số nghiệm nguyên dương của phương trình (*), áp dụng bài toán 1, ta có đáp số là: $C_{N+k-1}^{k-1} = C_{N+k-1}^N$

- Áp dụng kết quả của bài toán 2, ta có số nghiệm nguyên của các phương trình (1), (2), ..., (t) lần lượt là: $C_{N+x_1-1}^{x_1-1}, C_{N+x_2-1}^{x_2-1}, \dots, C_{N+x_t-1}^{x_t-1}$
- Vì việc chọn các bộ số cho mỗi thừa số phải được thực hiện đầy đủ, độc lập, không phụ thuộc thứ tự, không phụ thuộc nhau nên đáp số bài toán chính là tích đáp số của t bài toán nêu trên

Hướng giải (Tóm gọn):

1. Phân tích $M = p_1^{x_1} * p_2^{x_2} * \dots * p_t^{x_t}$

2. Đáp số bài toán chính là:

$$\prod_{i=1}^t C_{N+x[i]-1}^{x[i]-1}$$

Vấn đề về cài đặt tổ hợp $C(k, n)$:

- Ta biết rằng, $a^b = c$, với c không đổi, nếu a giảm thì b phải tăng và ngược lại
Mà $M = 10^9$, hỏi rằng số mũ to nhất trong phân tích thừa số nguyên tố của M là bao nhiêu. Số mũ to nhất, khi cơ số nhỏ nhất. Mà cơ số nhỏ nhất chính là 2
Ta giải phương trình $2^x = 10^9 \rightarrow x = \log_2(10^9) = 29.89$. Nên số mũ tối đa sẽ không đến 30, tức là $x[i] \leq 30$, với mọi i
- $N + x[i] - 1 = 10^5 + 30$. Như vậy, không thể xử lý mảng $C(k, n)$ 2D được
- Ta biết rằng $C_N^k = \frac{N!}{k! * (n-k)!}$. Như đã phân tích ở trên, $N_{\max} = 10^5, k_{\max} = 30$.
- Tiền xử lý $N! \bmod (MOD = 10^9 + 7)$: Gọi $fac[i] = i!$.
Đương nhiên $0! = 1, N! = (N-1)! * N. \rightarrow fac[i] = (fac[i-1] * i) \% MOD$;
Như vậy là đã xong giai thừa trên tử số
- Xử lý giai thừa dưới mẫu số:

Định lý Fermat nhỏ:

- p là số nguyên tố, a không chia hết cho p thì $a^{p-1} \equiv 1 \pmod{p}$
- Nhân cả 2 vế đồng dư thức với a^{-1} , ta được: $a^{p-2} \equiv a^{-1} \pmod{p}$

Áp dụng:

- Gọi $\text{ivFac}[i] = (i!)^{-1} = \frac{1}{i!}$
- Áp dụng định lý Fermat nhỏ, ta có: $\frac{1}{(\text{MAX})!} = (\text{MAX}!)^{\text{MOD}-2}$. Mà $(\text{MAX}!)$ theo modulo ta đã tính từ trước đó ($\text{fact}[\text{MAX}]$).
Vì số mũ rất lớn nên chỗ này sẽ sử dụng lũy thừa nhị phân để tính
- Từ đó, ta tính giật ngược nghịch đảo giai thừa về trước theo tính chất phân phối phép nhân theo modulo:

$$\frac{1}{i!} = \frac{1}{(i+1)!} * (i+1)$$

→ $\text{invFac}[i] = [\text{invFac}[i+1] * (i+1)] \% \text{MOD}$;

- Cuối cùng: $C_N^k = \text{fac}[n] * \text{ivFac}[k] \% \text{MOD} * \text{ivFac}[n-k]$

BÀI D. ĐƯỜNG TRÒN

- Cho đường tròn C tâm $I(X, Y)$, bán kính R .
- Một điểm $P(x, y)$ nằm trong C thì $IP \leq R \rightarrow (x-X)^2 + (y-Y)^2 \leq R^2$
- Như vậy, các điểm x khả dĩ là: $(x-X)^2 \leq R^2 \rightarrow \text{dist}X = |x-X| \leq R$
→ $-R \leq x-X \leq R \rightarrow X-R \leq x \leq X+R$. Đây chính là khoảng mà ta sẽ lặp X
- Với mỗi X như vậy, ta có $|y-Y| \leq \sqrt{R^2 - \text{dist}X^2} = \text{dist}Y$
→ $-\text{dist}Y \leq y-Y \leq \text{dist}Y \rightarrow Y-\text{dist}Y \leq y \leq Y+\text{dist}Y$.
- Cận trên luôn làm tròn lên, cận dưới luôn làm tròn xuống. Ta được Min, Max. Đáp án sẽ là: $\text{Max} - \text{Min} + 1$
- **Xử lý sai số: Đọc thêm bài viết:**
<https://www.facebook.com/share/p/17DGNmMpa2/>

BÀI E. NỐI ĐIỂM

- Về bản chất, bài toán này chính là tìm cây khung cực tiểu.
- **Nhắc lại về thuật toán Kruskal cơ bản:**
 1. Sắp xếp tập cạnh E theo trọng số tăng dần. Khởi tạo $MST_w = 0$, $cnt = 0$, với cnt là số cạnh cây khung
 2. Duyệt từng cạnh (u, v, w) trong E. Nếu nối (u, v) không tạo thành chu trình:
 - Nối (u, v) vào cây khung
 - $MST_w += w$, $cnt++$. Khi $cnt = \text{Số đỉnh} - 1$ thì dừng
- Các bạn đã học qua môn DSA ở PTIT chắc cũng biết đây là bài toán khó hơn của bài DSA10016 – Nối điểm. Bây giờ, ta sẽ cùng xét bài toán này
- Đối với bước kiểm tra chu trình, để cho thuận tiện thì ta sẽ sử dụng các hàm find và Union của CTDL DSU:

```
int find(int x){
    if(x == par[x]) return x;
    return par[x] = find(par[x]);
}
```

```
bool Union(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (sz[a] < sz[b]) swap(a, b);
    sz[a] += sz[b];
    par[b] = a;
    return true;
}
```

- Triển khai thuật toán:
 1. Nhập toạ độ N điểm, khởi tạo DSU: $par[i] = i$, $sz[i] = 1$
 2. Tạo tập cạnh khả thi:
 - Dùng vector để lưu cạnh.
 - Với mỗi cặp điểm (i, j) tính trọng số $dist(i, j)$ và đưa vào vector.
 3. Sắp xếp các cạnh trong v theo trọng số tăng dần
 4. Triển khai thuật toán Kruskal trên tập cạnh v đã sắp xếp
- Đối với bài này, N lên đến 10^5 , duyệt 2 vòng for như ver 1 thì lên đến 10^{10} vòng lặp, chắc chắn TLE.
- Phần kiểm tra chu trình, sử dụng DSU đã là tối ưu nhất, không cần tối ưu hơn. Ta cần xử lý chỗ tạo cạnh, sao cho kéo giảm ĐPT dưới $O(N^2)$.
- Đề cho biết tung độ (y) chỉ nằm trong đoạn $[0, 10]$.
⇒ Ta có thể gom các điểm theo tung độ.
⇒ Với mỗi tung độ Y, tập hợp các điểm chung Y lại rồi sắp xếp chúng theo X tăng dần
- Với một điểm X:
 - Không cần nối X với tất cả N - 1 điểm khác.

- Ta chỉ cần xét với các tung độ từ 0 đến 10.
- Trong mỗi tung độ, chỉ cần nối X với **2 điểm gần nhất** theo hoành độ (một bên trái, một bên phải).
- Để tìm nhanh 2 điểm gần nhất này:
 - Sắp xếp mỗi tập các điểm chung tung độ theo hoành độ.
 - Dùng **tìm kiếm nhị phân** để chọn láng giềng 2 bên cho điểm X
- Độ phức tạp chỉ còn $O(N \log N)$

BÀI F. CHIA HẾT CHO 8

Một số chia hết cho 8 khi và chỉ khi 3 chữ số cuối của nó chia hết cho 8. Do đó, ta chia thành 3 trường hợp chính:

TH1: Số có 1 chữ số: Bằng 8 thì chia hết, ngược lại thì không

TH2: Số có 2 chữ số: 1 trong 2 số là chính nó và số đảo của nó chia hết cho 8 là được

TH3: Số có 3 chữ số đảo lên:

- Chúng ta chỉ quan tâm 3 chữ số cuối.
- Giả sử số đó có dạng *******abc**:
 - $0 \leq a, b, c \leq 9$ và a, b, c nguyên
 - Phần phía trước không ảnh hưởng đến tính chất chia hết cho 8, ta không cần xét
- Sinh 3 vòng for cho các chữ số a, b và c . Mỗi số đều duyệt từ 0 đến 9.
Lúc này, giá trị của đuôi chính là $V = 100a + 10b + c$. Nếu V chia hết cho 8, đây là một bộ ba khả thi
- Kiểm tra xem chuỗi gốc có đủ số chữ số để tạo ra bộ ba abc này không:
 - Duyệt tất cả các chữ số trong xâu gốc, lưu vào mảng $cnt[10]$. Với $cnt[i]$ là số lượng chữ số i
 - Duyệt 3 chữ số a, b, c để đếm số lượng số cần cho bộ 3 này, lưu vào $needed$.
 - So sánh $needed[d]$ với $cnt[d]$ cho mọi chữ số d .
Nếu $cnt[d] \geq needed[d]$ với tất cả d , thì bộ ba này có thể tạo ra từ chuỗi.
Ngược lại, bộ ba không khả thi.
- Tổng số trường hợp cần xét: 3 vòng lặp cho a, b và c : 10^3 , mỗi bộ ba thêm 10 lần so sánh. ĐPT chung là 10^4